РЕКЛАМА

antipov_dmitry 29 окт 2025 в 12:18

Выбираем векторную БД для AI-агентов и RAG: большой обзор баз данных и поиск смысла

18 мин 17K

Базы данных*, Искусственный интеллект, Машинное обучение*, Анализ и проектирование систем*

В этой статье я сделал обзор основных векторных баз данных: Milvus, Qdrant, Weaviate, ChromaDB, pgvector, Redis, pgvectorscale, LanceDB, ClickHouse, Vespa, Marqo, ElasticSearch.

Если вы запутались в разнообразии векторных баз данных или хочется верхнеуровнево понимать как они устроены, чем отличаются и для чего вообще нужны, то эта статья будет очень полезна. Мы пошагово соберем все ожидания от векторных БД, посмотрим бенчмарки, а затем попробуем собрать все воедино.



А ты выбрал базу для RAG или AI-агентов?

Звучит немного тавталогично, но на самом деле выбор подходящей векторной базы начинается даже до момента непосредственно самого выбора базы данных. Потому что очень важно не только выбрать ГДЕ хранить, но и ЧТО. Векторные БД хранят вектора (сюрприз), но сами вектора — явление широкое и потому про них стоит сделать важное вступление.

Поиски смысла и вектора (эмбединги)

Да, если вы все это хорошо знаете, то можно переходить непосредственно к бенчмаркам, но эта часть — супер важная.

Эмбединги (embeddings) — это числовые векторные представления данных (текста, изображений, аудио, кода, видео и чего угодно), полученные с помощью специализированной модельки, которая называется энкодером (или эмбеддером). Такая модель — фактический «запаковщик смысла»: она преобразует любые входные данные в массив чисел фиксированной длины (вектор), где семантически похожие объекты располагаются близко друг к другу в многомерном пространстве.

Пример: фраза «клиент просил вернуть деньги» преобразуется в условный вектор [0.23, -0.15, 0.89, ...], фраза «заказчик хотел получить возврат средств» в вектор [0.21, -0.14, 0.87, ...]. Косинусное сходство *между векторами — 0.95, то есть обе фразы очень близки по смыслу, хотя и написаны совершенно разными словами. Если мы возьмем «котик прыгает по столу», то будет другой вектор, маленькое косинусное* сходство *и, как следствие, очень далекий смысл от первых двух.*

Эмбединг = смысл. Но все мы работаем с разными смыслами и это — большая проблема. Понятие смысла субъективно и сильно зависит от предметной области. Даже под одними и теми же словами можно подразумевать разное: реверс (двигателя, монеты или инжиниринг?), артефакт (искажение или объект?) или же относиться к самым разным индустриям (например, слово «протокол» — оно про IT, медицину или криминалистику?).

Поэтому эмбединги, а точнее, модели которые их генерируют, можно условно разделить на две большие группы: общего назначения (обученные на всем корпусе языка) или же специализированные. Конечно, специализированные будут работать лучше: если мы делаем медицинский RAG, то «*аспирин*» должен скорее ассоциироваться с «*циклооксигеназа-1*» , чем с просто головной болью. Но специализированные модели — недешевое во всех смыслах удовольствие.

У эмбедингов есть размерность (к ней мы еще вернемся) и, если упростить, чем она больше, тем больше информации в него зашито, но и тем больше памяти для этого требуется.

И векторные базы данных предназначены для хранения этих самых эмбедингов. Но не только хранения, но еще и различных операций.

Давайте сначала поговорим про само хранение.

Реклама удалена.
Показать дополнительные сведения

Хранение эмбедингов в БД

Эмбединг — это сжатое числовое представление чего угодно, например, кусочка текста или видео. Мы оперируем этими векторами потому что над ними удобно проделывать математические операции, но в конечном итоге нам нужны исходные данные. Например, мы по векторам уже нашли 10 похожих видео на наше, но мы не можем отдать эти вектора клиенту, нам нужны именно видосики.

Поэтому помимо самого хранения эмбединга нужно хранить мета-информацию — кем загружно, категорию, название, сам файл и так далее. И тут есть два пути:

1. Хранение в самой векторной БД (встроенное)

```
{
  "id": "doc_123",
  "vector": [0.1, 0.2, ...],  // 768 чисел
  "metadata": {
    "text": "Полный текст документа или чанка",  // ← оригинал здесь
    "title": "Название",
```

```
    "date": "2025-07-05",
    "category": "tech"
  }
}
```

Объяснить с

Из плюсов — одна система, одна точка отказа, атомарность операций, минусы — векторные БД в первую очередь оптимизированы под вектора и хранить большое количество мета-информации будет дороже по I/O и латенси. Такое уместно в небольших RAG-системах (<1M документов), прототипы или системы с короткими чанками (<1000 условных токенов)

2. Внешнее хранилище + reference

Векторная БД хранит только эмбединги, ID и опционально — краткие метаданные, а все оригиналы — в привычной базе. Из плюсов здесь то, что каждый инструмент делает свою работу, из минусов — нужна синхронизация.

Есть еще один частный способ в виде гибридного решения: вектор в PostgreSQL, blob в S3.

С хранением — плюс-минус разобрались, теперь — с операциями.

Операции над эмбедингами

Основная операция — **векторный поиск** (ANN search): «найди K наиболее похожих векторов на запрос». Это *не точное* совпадение как в SQL (`WHERE id = 5`), а поиск

по близости — топ-10, топ-100 самых семантически близких объектов. Результат возвращается по мере близости/расстояния, отсортированный по релевантности.

CRUD-операции: Insert (добавить новые векторы), Update (обновить метаданные, сам вектор обычно не меняется), Delete (удалить), Upsert (обновить или создать, если не существует). При вставке вектор автоматически добавляется в индекс для быстрого поиска. Про индекс мы еще поговорим.

Фильтрация по метаданным: векторный поиск + условия на метаданные в одном запросе. Например: «найди похожие документы, но только category='tech' AND date >= 2025". Критически важно, чтобы фильтры применялись **во время** поиска (pre-filtering), а не после (post-filtering), иначе теряется точность.

Гибридный поиск: например, комбинация векторного поиска + полнотекстового (BM25) + фильтрация метаданных. Векторный находит по смыслу, BM25 усиливает точные ключевые слова, фильтры отсекают по метаданным.

Batch-операции: массовая загрузка тысяч/миллионов векторов за раз (bulk insert), массовое удаление, переиндексация. Критично для первоначальной загрузки больших датасетов или периодической синхронизации.

Реиндексация: пересоздание индекса с другими параметрами (тип индекса, метрики сходства/расстояния, квантизация) без перезагрузки оригинальных векторов. Позволяет экспериментировать с настройками производительности.

Все эти операции должны работать быстро даже на миллионах векторов — это главное отличие специализированных векторных БД от «просто хранилища массивов чисел».

Что общего у всех векторных БД

Базовая архитектура векторных БД похожа на реляционные, но с векторной спецификой:

- **Коллекции** (collections/tables) — аналог таблиц в SQL, каждая коллекция хранит векторы одной размерности.
- **Документы/точки** (points/documents) — аналог строк. Каждый документ содержит: вектор (массив чисел) + метаданные (JSON-подобная структура с категориями, датами, тегами и тд)
- **Индексы** — обязательная структура для быстрого поиска, про них мы поговорим
- **Метрики близости вместо JOIN'ов.** Векторные БД используют математические меры для поиска: меры сходства (больше = лучше) — Cosine Similarity и Dot Product, и меры расстояния (меньше = лучше) — L2/Euclidean. Метрика фиксируется при создании коллекции — смена требует переиндексации
- **Векторный поиск** вместо SELECT. Базовая операция: «найди K ближайших векторов к запросу» (вместо «выбери строки WHERE условие»). Результат — отсортированный список по similarity score

Отличие от реляционных БД: вместо точного поиска (WHERE id = 5) — приблизительный поиск (топ-10 похожих). Вместо нормализации (связи между таблицами) — денормализация (всё в одном документе). Многие векторные БД ориентированы на масштабируемость, и поведение согласованности может быть менее строгим, чем у классических реляционных БД.

Индексы в векторных БД

ЧИТАЮТ СЕЙЧАС

Я позволил ИИ отслеживать мой телефон всю неделю. Вот что я потом узнал о себе

21K 16 +16

Российский мессенджер MAX замечен в обращении к иностранным сервисам определения IP и серверам конкурентов

32K 82 +82

Маркетплейсы убили мои продажи — а теперь я зарабатываю 1,7 млн, печатая

Я упомянул слово «индекс» — они для векторов бывают разные, и от выбора индекса зависит скорость поиска, потребление памяти и точность результатов:

- **HNSW (Hierarchical Navigable Small World)** — многослойный граф навигации, золотой стандарт для скорости и точности
- **IVF (Inverted File Index)** — семейство индексов, разбивает пространство на кластеры, быстрая индексация и работа с диском
- **DiskANN** — индекс для SSD (NVMe), позволяет работать с терабайтами векторов при минимальной RAM
- **Flat (Brute Force)** — точный поиск без индекса, сравнение со всеми векторами, используется для малых датасетов (<100K)
- **Product Quantization (PQ)** — сжимающий индекс, экономит память в 8-32 раза за счет небольшой потери точности

Каждая векторная БД не просто поддерживает свой набор индексов, а *оптимизирует их под свое хранилище и движок* — например, Qdrant модифицировал HNSW для лучшей работы с фильтрами, а Timescale создал свой StreamingDiskANN специально под PostgreSQL. Эти оптимизации дают преимущество в 20-40% по производительности против универсальных реализаций.

Квантизация индексов — еще один способ оптимизации. Векторы хранятся в float32 (4 байта на число), а квантизация сжимает их до int8 или даже битов. БД со встроенной квантизацией (Qdrant, Milvus, pgvectorscale) позволяют экспериментировать: загрузили векторы один раз (они хранятся как есть) → пересоздаем индекс с разной квантизацией → сравниваем память/скорость/точность. Оптимизированные под движок версии могут работать гораздо лучше самописных —

на их товаре

46K 28 +28

Запущен аналог Cursor из России

14K 35 +35

Apple официально представила MacBook Neo — ноутбук за 600 долларов на базе мобильного чипа A18 Pro

4.4K 21 +21

ИИ здесь, ИИ там, ИИ везде? Опрос

Турбо

например, Binary Quantization от Qdrant с oversampling *по их заявлениям* дает recall 95%+ против 70-80% у наивной реализации.

Это было перечисление, скажем так, физических операций и оптимизаций в них, а теперь попробуем составить требования именно под RAG и агентов.

Операции с БД в RAG и AI-агентах

RAG

Суть RAG — это поиск, а «R» в аббревиатуре в сто раз важнее «G». RAG так хорошо зашел не потому, что это новая технология, а потом что LLM умеют разрозненную информацию единообразно подать.

Процесс: документы разбиваются на фрагменты, каждый преобразуется в эмбединг; запрос пользователя векторизуется и ищутся похожие фрагменты; найденные фрагменты передаются LLM как контекст.

Задачи векторной БД в RAG: семантический поиск (основная задача), фильтрация метаданных (например, дата, автор, тип документа), гибридный поиск (векторный + BM25), переранжирование результатов (reranking), работа с большим объемом документов, инкрементальная индексация.

AI-Агенты

Агенты используют векторные БД иначе: долгосрочная память (история взаимодействий), планирование действий (поиск примеров из прошлого) и динамический контекст.

Их специфические требования:

- Быстрые вставки (агенты часто пополняют базу)
- Версионирование (отслеживание изменений знаний)
- Множественные коллекции (разные типы памяти: эпизодическая, семантическая, процедурная)
- Транзакционность для согласованности

Ну, кажется, можно переходить к кандидатам.

Бенчмаркинг векторных БД и VectorDBBench

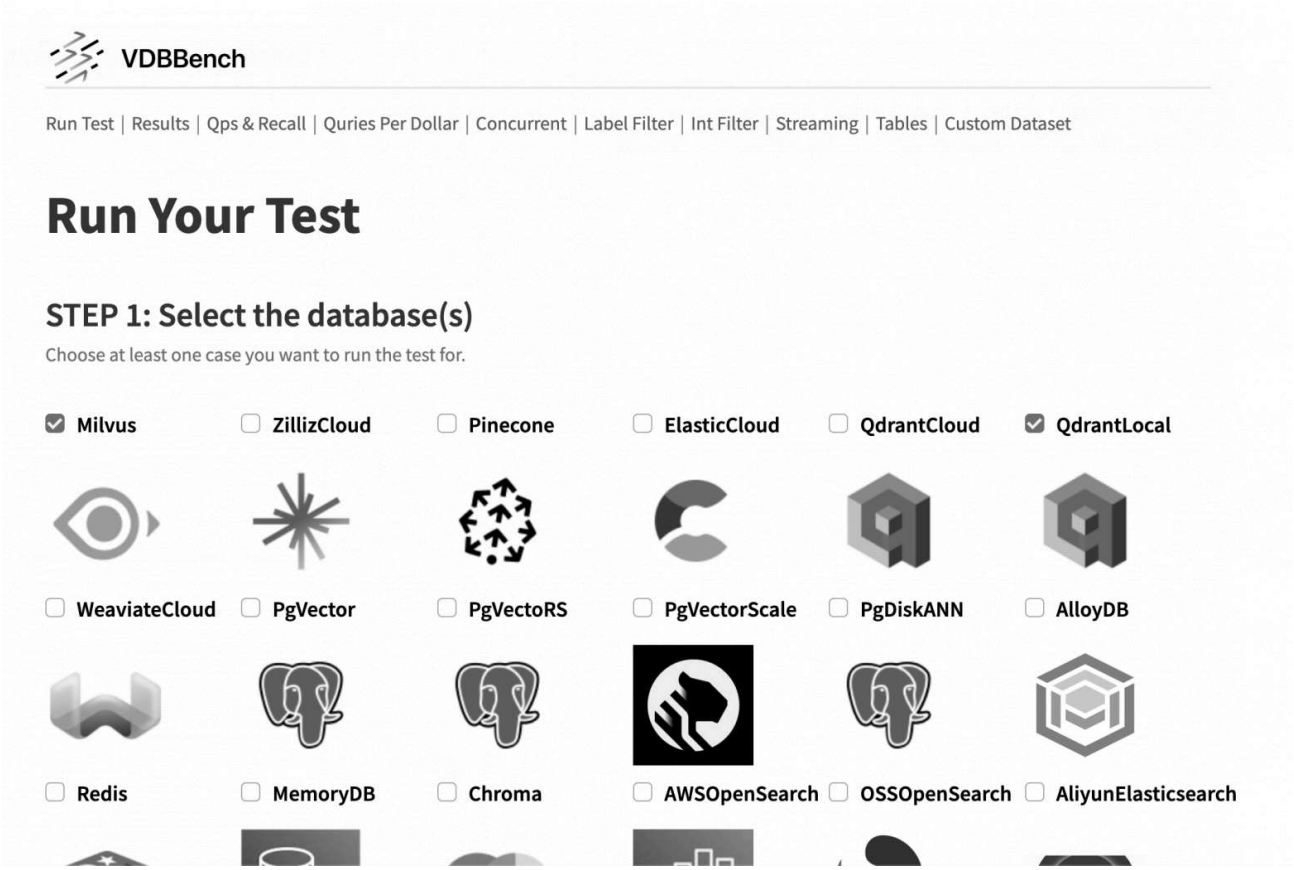
Когда я вообще делал первый подход к снаряду выбора векторной БД, то мне показалось уместным пойти посмотреть публичные бенчмарки. Но в публичных бенчмарках от самих разработчиков часто оказывалось так, что их решение — лучшее. А значит у нас есть примерно 15 лучших баз данных. *[можно выбрать любую из них — расходимся, хаха]*

В целом, скорее всего, там никто не врет, но у всех разные данные, разное железо и немного разные способы подсчета.

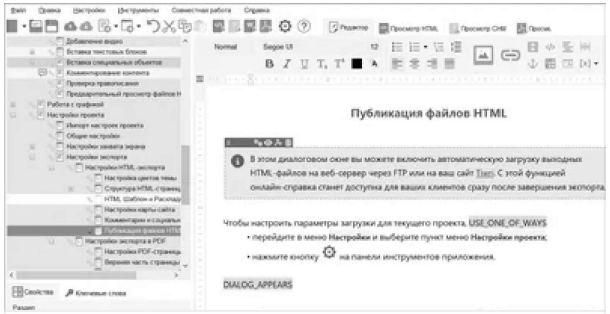
А если в публичные бенчи мы не верим, значит надо делать свой, ведь других-то у меня для вас нет. Или есть? В бездонных просторах ресерчей я нашел инструмент, который придумали разработчики одной популярной векторной БД, и который подхватило векторное коммьюнити. Имя ему — VectorDBBench. И это очень крутая

штука, которая позволяет сделать фиксированные тесты не на какой-либо там синтетике.

Ставится себе на локалку и выглядит вот так:



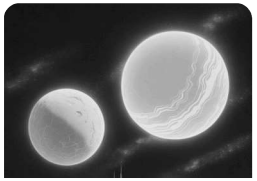
Круто? Круто. Выбираем БД, выбираем датасет (можно свой или публичный), запускаем тест и получаем все классные метрички типа QPS, latency (P50/P95/P99), recall, indexing time.



Dr.Explain — не Word: делать руководства и мануалы проще

drexplain.ru РЕКЛАМА • 16+

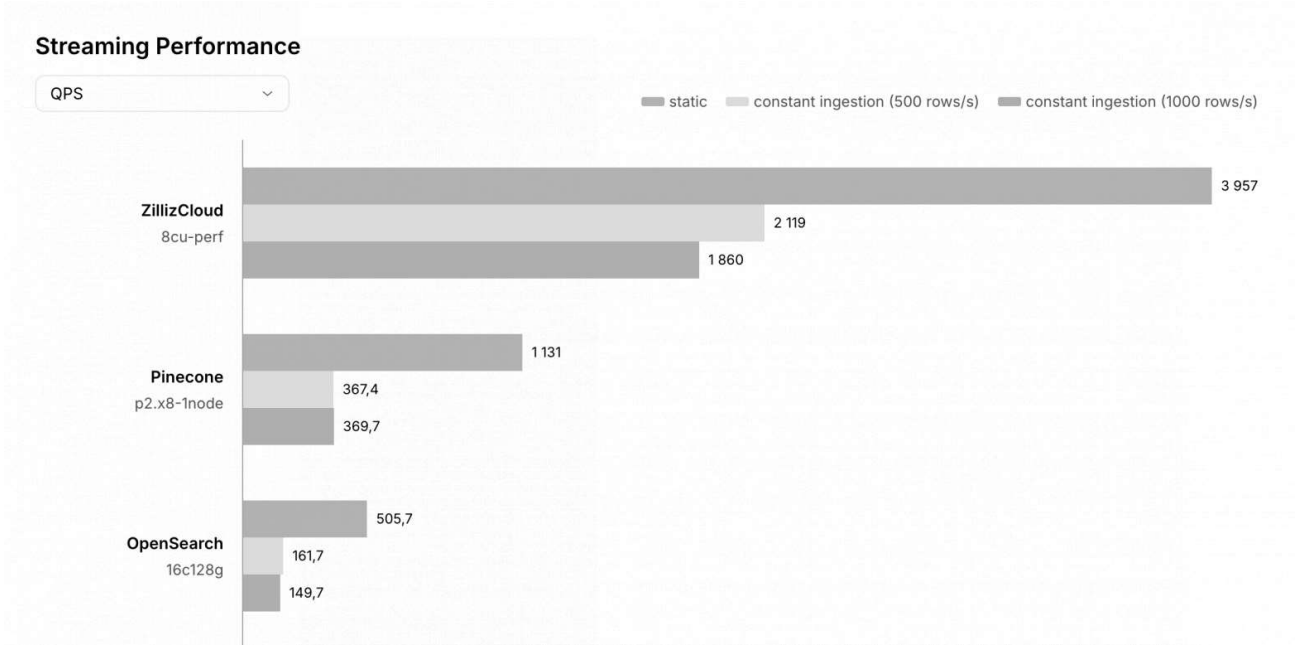
ИСТОРИИ





Инструмент открытый, методология открытая, если уже есть данные и встает вопрос выбора векторной БД, то прогнать всевозможные тесты здесь — правильный и скорее обязательный путь.

Все очень классно сделано — то, что называется «люди подумали». Единственное, по их публичной стате в топ почти всего часто влетает ZilizCloud (их протюненная облачная версия Milvus), что, возможно, намекает на то что они все же под бенчмарк оптимизировались. Но наличие официальных контрибьюторов из других БД доверие все же внушает, плюс, опять же, все тесты надо делать на своей инфре, своих задачах и своих данных — только так оно будет что-то реально показывать.



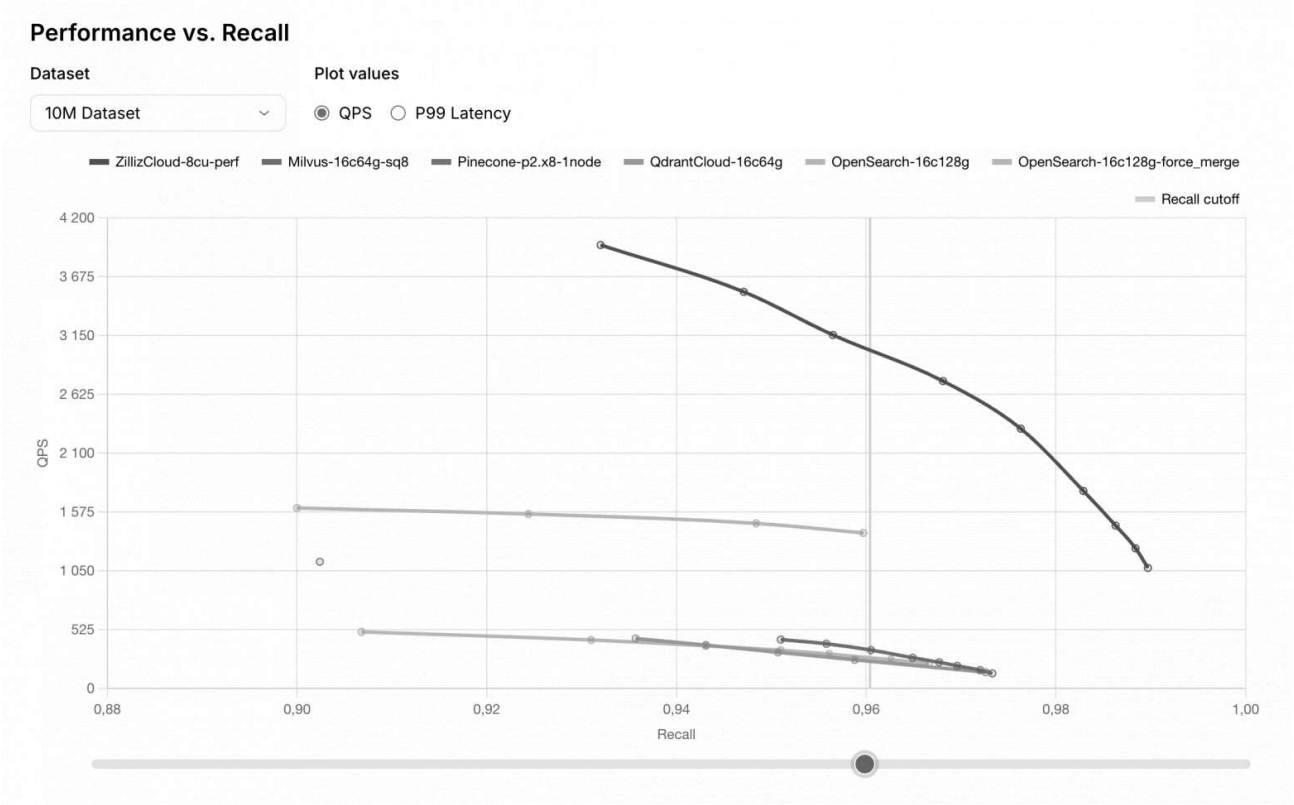
Streaming Performance



Лучшее из блогов компаний за неделю



Кот в доме хозяин



Performance vs Recall

Типы тестов в VectorDBBench

1. **Search Performance Test** — чистый векторный поиск без фильтров. Базовая метрика для всех БД
2. **Int-Filter Search Performance Test** — векторный поиск + числовые фильтры (year >= 2020, price < 1000), показывает деградацию при фильтрации
3. **New-Int-Filter Search Performance Test** — сложные комбинации числовых фильтров, имитирует production-запросы

4. **Label-Filter Search Performance Test** — векторный поиск + категориальные фильтры (category="tech", status="active"), выявляет post-filtering vs pre-filtering
5. **Capacity Test** — максимальное количество векторов до критической деградации., показывает реальные лимиты масштабируемости
6. **Streaming Test** — производительность поиска при непрерывной вставке новых данных, критично для систем с постоянными обновлениями
7. **Custom Search Performance Test** — тестирование на наших данных с нашими паттернами запросов

Максимально рекомендую для серьезных исследование, а мы переходим к самим БД!

Список векторных БД под микроскопом

Надеюсь, я донес основную идею, что все очень сложно и специфично и сначала нужно декомпозировать задачу, зафиксировать способ получения эмбедингов и именно под это уже выбирать БД. Поэтому пересказывать публичные бенчи смысла нет, а вот пройтись по каждой базе и ее оценить — это правильно.

Продраться сквозь маркетинговые заявления и сравнивания баз между собой на их бенчмарках было непросто, но оно того стоило.

> Здесь стоит учесть, что зафиксировать в моменте все плюсы/минусы, масштабы и сложность — задача крайней сложная, потому что векторные базы — тема горячая и почти все продукты ниже развиваются **очень быстро**. Люди делают бенчмарки, пишут обзоры, а через месяц выходит обновление, которые делает

> все сделанное устаревшим. Поэтому список и таблицу стоит читать как ориентир и некую точку фиксации развития векторных БД. В описании и таблице *возможны* локальные неточности, я буду очень рад их исправить.

Да, из списка выброшены все базы, у которых есть vendor-lock (фактически cloud-based), просто потому что в 2025 это очень важный фактор выбора БД.

1. Milvus

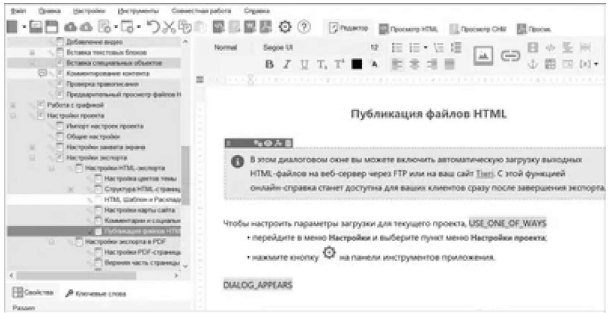
Философия:

- Открытая распределенная векторная БД, поддерживает масштабирование кластеров, GPU-ускорение, множество индексов (HNSW, IVF, PQ и др)
- Спроектирована для управления большими объемами векторов («миллиарды точек») и поддерживает гибридные сценарии

Плюсы:

- Хорошая масштабируемость (от себя до больших кластеров, распределенность)
- Поддержка разных режимов индексов и оптимизаций (GPU, диск-индексы)
- OSS-лицензия Apache 2.0
- Подходит для серьезных RAG / AI-агентов с большим объемом данных

Минусы:



Dr.Explain — не Word: делать руководства и мануалы проще
drexplain.ru РЕКЛАМА · 16+

БЛИЖАЙШИЕ СОБЫТИЯ



- Более высокая сложность настройки и эксплуатации (особенно кластер)
- Требует серьезного инфраструктурного ресурса при больших нагрузках *(но, с другой стороны, если вы до них доросли, то может оно и ок?)*
- Может быть оверкилом если объем данных невелик и требования просты

Идеально для: Сценариев, когда объём данных велик (сотни миллионов-миллиарды векторов), нужна высокая пропускная способность и гибкость (например, мультимодальные данные, видео/изображения + текст), возможно собственный хостинг, и когда команда готова управлять кластером.

2. Qdrant

Философия:

- OSS-векторная БД с акцентом на payload-фильтры (метаданные) + векторный поиск
- Поддержка индексов, фильтрации, гибридных запросов, оптимизаций под реальные приложения

Плюсы:

- Хороший баланс между производительностью и гибкостью
- Легкость старта (по сравнению с крупными кластерами)
- Сильная поддержка фильтрации по метаданным + векторов — важно для RAG

26 февраля – 6 марта

«Ситуация — 8 марта»: истории из офиса

Онлайн

Другое

Больше событий в календаре

Минусы:

- Масштабируется, но уступает решениям по very large scale сценариям (например, миллиард+ точек) в плане масштаба или функциональной экосистемы
- Управление кластером требует усилий

Идеально для: проектов среднего масштаба, где нужен семантический поиск + метаданные-фильтры, например RAG-сценарии, логика агентов (история, память) с числом точек в диапазоне миллионов-десятков миллионов.

3. Weaviate

Философия:

- OSS-векторная БД с «knowledge graph / schema» подходом: позволяет задавать схемы, модули встраивания, гибридный поиск (ключевые слова + векторы)
- Предлагает GraphQL API + модуль для генерации эмбеддингов внутри (опционально) — упрощает интеграцию

Плюсы:

- Очень удобно для сложных структур данных, где нужна не просто векторная точка, а богатая схема, связи, категории
- Гибридный поиск «вектор + ключевые слова» встроен

- Подходит для RAG/агентов с метаданными и сложными связями

Минусы:

- Может быть более сложной в настройке, если вы просто хотите «хранить и искать векторы» без схемы
- При очень больших объёмах и сверхвысоких требованиях по пропускной способности и может уступать специально оптимизированным решениям
- Возможна избыточность функционала, если требования просты

Идеально для:
Сценариев, где данные имеют богатую структуру (например, знания, графы, сложные связи): RAG системы с метаданными, AI-агенты с памятью, где нужна не только «найти похожий фрагмент», но «найти похожий + связанный + в фильтре».

4. ChromaDB

Философия:

- Легкая и удобная векторная БД для прототипов, небольших проектов.
- Часто используется как стартовое векторное хранилище, меньше фокус на «миллиарды точек», больше на «быстро запустить»

Плюсы:

- Простота, скорость старта, низкий порог входа
- Отлично подходит для прототипов, экспериментов, RAG PoC
- Обычно легче интегрировать, меньше инфраструктурной нагрузки

Минусы:

- Менее производительная или масштабируемая по сравнению с крупными решениями

Все потоки 💰 Узнали, сколько получают в IT Войти

  точек, или с высокими требованиями к пропускной способности/латенси
- Возможны ограничения в функциональности (индексы, масштабирование, enterprise-фичи)

Идеально для:
Прототипов, PoC RAG/агентов, стартапов, когда число документов/векторов ещё умеренное (например < 10 м), когда важна скорость разработки, а не максимальная производительность или распределенный кластер.

5. pgvector (расширение для PostgreSQL)

Философия:

- Расширение для PostgreSQL, которое позволяет хранить и искать векторы внутри знакомой реляционной БД (SQL) среды

- Подходит когда уже используется PostgreSQL и хочется добавить семантический поиск без отдельной системы

Плюсы:

- Используется уже существующая инфраструктура, знакомая среда
- Нет необходимости внедрять новую систему, команда может продолжать работать с SQL + векторы
- Отлично для интеграции структурированных данных + векторов (гибридные сценарии)

Минусы:

- Не оптимизирован под большие масштабы векторного поиска, особенно с десятками миллионов точек/низкой задержкой
- Может уступать специализированным движкам по скорости, индексации, масштабированию
- Функционал может быть более ограниченным по сравнению с чистыми векторными БД

Идеально для:
Когда основная база данных — PostgreSQL, векторный поиск — дополнительная функция, объемы данных умеренные, а требования к латенси не экстремальные. Например, небольшие RAG-модули внутри существующей системы.

6. Redis

Философия:

Redis в новых версиях (≥ 7) имеет модуль Redis Search с поддержкой векторного поиска по float-векторам. Поиск выполняется через HNSW индекс.

Плюсы:

- Быстрая in-memory работа
- Простая установка, много SDK

Минусы:

- Хранение в RAM — дорого для больших объемов
- Менее эффективен для дисковых данных
- Не подходит для терабайтных векторных архивов

Идеально для:
Быстрого кеширования векторов, AI-агентов с памятью, мгновенного поиска на небольших датасетах (< 10 M векторов).



7. pgvectorscale

Философия:

Расширение для PostgreSQL, разработанное Timescale для ускорения векторного поиска на базе pgvector. Добавляет собственные индексы (StreamingDiskANN), оптимизированные для больших объемов векторов и дисковых операций.

Плюсы:

- Высокая скорость поиска по сравнению с vanilla pgvector
- Простая интеграция в существующие SQL-схемы
- Поддержка гибридного поиска (вектора + WHERE условия)

Минусы:

- Требует наличия и настройки PostgreSQL и Timescale
- Масштабирование — через шардинг, не встроено автоматически
- На миллиардах векторов проседает по производительности чистым векторным БД

Идеально для:
Если уже есть PostgreSQL, и хочется добавить векторный поиск без нового технологического стека. Отличный вариант для встраивания в существующие продукты, внутренние сервисы или умеренные RAG-нагрузки.



8. LanceDB

Философия:

Open-source векторная БД, основанная на Apache Arrow и формате Lance.
Оптимизирована для мультимодальных данных (текст, изображения, аудио).

Плюсы:

- True vector store с простым Python API
- Использует колоночное хранилище (Arrow + Lance) для скорости и низких затрат
- Можно хостить у себя в локальных ноутбуках

Минусы:

- Молодая экосистема, меньше обвязки для кластеров
- Не так масштабна, как Milvus

Идеально для:
Data-science RAG, AI-агентов, которые работают локально или в небольших кластерных сценариях. Идеально для Jupyter / LangChain экспериментов.

9. ClickHouse

Философия:
Колоночная аналитическая БД с добавленной поддержкой векторного поиска (HNSW и Flat). Хранит векторы в столбцах и индексирует их.

Плюсы:

- Масштабируемость и скорость аналитических запросов.
- Поддержка гибридных запросов (векторы + SQL + агрегации)

Минусы:

- Не чисто векторная БД, а скорее SQL + векторы
- Настройка индексов чуть сложнее, чем в других бд
- Требует понимания ClickHouse архитектуры

Идеально для:
Систем аналитики или RAG, где нужно объединять векторный поиск и SQL аналитику.
Отлично подходит для логов, метрик, векторных данных в одном месте.

10. Vespa

Философия:

- Vespa — открытая платформа «search engine & vector DBMS», поддерживающая поиск по векторным эмбдингам + текст + структурированные данные
- Поддерживает ANN-поиск, графы HNSW, масштабирование на кластер, мультивекторные/мультимодальные сценарии

Плюсы:

- Высокая гибкость: можно комбинировать векторный поиск + обычный full-text + структурированные фильтры
- Подходит для больших масштабов («billion-scale vector search») и серьезных систем

Минусы:

- Настройка и эксплуатация могут быть сложнее привычного
- Требует инфраструктуры, знания архитектуры распределенных систем
- Возможно избыточна, если данные/нагрузка невелики

Идеально для:
Сценариев, когда нужен серьезный production, большая нагрузка, комбинированный поиск (векторы+текст+структуры), self-host возможность. Например, RAG/агенты с большим объёмом данных, где важны фильтры, гибридный поиск, масштабирование.

11. Marqo

Философия:

- OSS векторный поисковый движко/хранилище, ориентирован на мультимодальные данные (текст + изображения) и делает «document in, document out» упрощая весь цикл: эмбеддинги, хранение, поиск
- Поддерживает хранение и поиск векторов, упрощает разработку (абстрагирует часть инфраструктуры)

Плюсы:

- Легко начать, упрощенная интеграция (особенно для мультимодальных данных)
- Хороший выбор, если нужен быстрый старт с векторным поиском, включая изображения + текст

Минусы:

- Экосистема менее зрелая, чем у лидеров
- Возможно меньше возможностей масштабирования/оптимизаций по сравнению с топовыми решениями
- Может быть ориентирована больше на простой старт, чем максимальную производительность на большом масштабе

Идеально для:
Прототипов или production-решений с умеренным масштабом, где нужно мультимодальное хранение/поиск, и когда хочется минимизировать инфраструктурную нагрузку. Например стартапы, экспериментальные RAG/агенты с изображениями + текстом.

12. ElasticSearch

Философия:

- Elasticsearch (и семейство Lucene) давно поддерживает векторный поиск (через поле `dense_vector`, kNN, ANN) и может использоваться как векторное хранилище
- Он также является полнотекстовым поиском + аналитикой, что дает гибридные возможности: текст + векторы + фильтры

Плюсы:

- Если у вас уже есть Elasticsearch, можно добавить векторный поиск без дополнительной системы
- Отлично подходит для гибридных сценариев (фильтры, текст, векторы)
- Взрослая экосистетма, множество инструментов, мониторинг, масштабирование

Минусы:

- Возможно не так оптимизирован под исключительно крупномасштабные векторные случаи как специализированные векторные БД
- Настройки индексов векторов могут быть сложны, производительность/точность может требовать тюнинга
- Может быть тяжелым по ресурсам, если используется только как векторное хранилище

Идеально для:
Сценариев, где уже используется Elasticsearch и хочется добавить

семантичесческий/векторный поиск без кардинальной смены стека. RAG с высоким уровнем полнотекстового поиска + векторы + фильтры — хороший вариант.

Сводная таблица векторных БД

БД (сложность + масштаб)	Характеристики	Идеально для
Milvus ☆☆☆☆☆ 1B+	**Плюсы**: GPU-ускорение, множество индексов (HNSW/IVF/DiskANN/GPU), доказанный масштаб, огромное количество production кейсов **Минусы**: Сложная настройка (etcd+MinIO+Pulsar), требователен к ресурсам на объеме, overkill для малых данных	Production с миллиардами векторов, GPU-ускорение, мультимодальные данные, есть DevOps команда
Qdrant ☆☆ 10M-100M+	**Плюсы**: Лучшая фильтрация (pre-filtering), Rust надёжность, один исполняемый файл, оптимизация памяти	RAG с фильтрацией, 1M-10M векторов, простота развертывания критична

	Минусы: Деградация на больших объемах без шардинга	
Weaviate ☆☆☆ 100M-1B+	**Плюсы**: GraphQL API, граф знаний, гибридный поиск (BM25+векторы) **Минусы**: Медленнее для чистого векторного поиска, сложные графовые запросы дорогие	RAG с контекстными связями, GraphQL API, богатые метаданные
ChromaDB ☆ <1M	**Плюсы**: `pip install` , нулевая конфигурация, LangChain/LlamaIndex first-class **Минусы**: Single-node, оптимальный эмпирический лимит ~1M без доп тюнинга, нет RBAC/HA, высокое потребление памяти	Прототипы, PoC, обучение, <500K векторов, быстрые demo
pgvector ☆☆ 1M-1B+	**Плюсы**: SQL интеграция, PostgreSQL инфраструктура, ACID для метаданных, любое масштабирование через Citus	Уже PostgreSQL, <10M векторов, ACID критичны

	Минусы: Медленная индексация	
Redis Stack ☆☆ <100M	**Плюсы**: Задержка <10ms, in-memory скорость 16x (v7.4), протестирован на 1B, поддерживает некое on-disk ANN (векторы на диске, индекс в памяти) **Минусы**: Дорого (in-memory)	Задержка <10ms критична, реал-тайм, кеширование
pgvectorscale ☆☆☆ 10M-100M	**Плюсы**: на порядок быстрее pgvector *(по их заявлениям)*, StreamingDiskANN **Минусы**: шардинг ручной *[здесь было упоминание, что обязательно требуется Timescale, но нет, это не требование, а рекомендация, минус не актуален]*	PostgreSQL + векторы, оптимизация затрат, альтернатива managed
LanceDB ☆☆ 1B+	**Плюсы**: 500M+ на одном узле, память 0.1-0.2GB на 1M (disk), zero-copy версионирование, S3/GCS direct	10M-1B+ на одном узле, serverless, встроенные приложения, cost-sensitive

	Минусы: Молодая экосистема, в основном IVF-PQ, нет кластера	
ClickHouse ☆☆☆ 100M-500M	**Плюсы**: SQL аналитика + векторы, гибридные запросы, колонночное хранилище **Минусы**: Не чисто векторная БД, сложная настройка индексов	Аналитика + векторы, логи/метрики + поиск в одном месте
Vespa ☆☆☆☆ 1B+	**Плюсы**: Billion-scale proven, гибридный (векторы+текст+структура) **Минусы**: сложная настройка, тяжелая инфраструктура, крутая кривая обучения	Enterprise production, большие нагрузки, комбинированный поиск
Marqo ☆ <10M	**Плюсы**: Мультимодальность (текст+изображения) из коробки, «document in, document out» **Минусы**: Менее зрелая экосистема, ограничения масштабирования	Прототипы с мультимодальностью, текст+изображения RAG

ElasticSearch ☆☆☆☆ 100M-500M	**Плюсы**: Mature ecosystem, гибридный (full-text+векторы+фильтры), богатый инструментарий **Минусы**: Не оптимизирован для только векторов, тяжёлый по ресурсам	Уже используется ES, гибридный поиск + аналитика критичны

FAQ: векторные БД для RAG и AI-агентов

1. Делаю RAG на 6M документов, что выбрать?

Qdrant — если с нуля (простое развертывание, отличная фильтрация).
pgvectorscale — если уже PostgreSQL (конкурентен с Qdrant на 10M-100M). У ChromaDB эмпирический лимит 1M, а Milvus — сложно.

2. Что выбрать чтобы обкатать идею?

ChromaDB — `pip install chromadb` , три строки кода и все работает. Идеально для PoC, MVP, экспериментов и демок за выходные. Как только идея зашла и >500K векторов — можно мигрировать на Qdrant или pgvectorscale.

3. У меня PostgreSQL/Redis, нужна ли отдельная векторная БД?

PostgreSQL + pgvectorscale: годится если <100M векторов, нужны JOIN'ы/ACID, команда знает Postgres. Отдельная БД если >100M, задержка <20ms критична, медленная индексация не подходит.

Redis Stack: годится если <100M векторов, критична задержка <10ms, данные помещаются в RAM. Отдельная БД если данные не влезают в память (дорого) или нужны >100M векторов.

Гибридный паттерн (типичный production): структурированные данные в PostgreSQL/Redis, векторы в Qdrant/Milvus, связь по ID.

4. Нужна ли квантизация в коробке?

Да если: >1M векторов, не хватает RAM, размерность ≥768D. **Нет** если: <500K векторов или максимальная точность критична.

5. Нужен ли гибридный поиск (векторы + BM25)?

Да если: точные термины важны (коды товаров, номера ошибок, названия), запросы содержат редкие слова. **Нет** если: общие для речи запросы, точные совпадения не критичны. Типичные веса: 70% векторный + 30% BM25. Лучшая поддержка: **Weaviate, ElasticSearch, Vespa.**

6. Какую БД выбрать для AI-агента с долгосрочной памятью?

Qdrant — лучший выбор для агентов. Pre-filtering метаданных (timestamp, session_id, user_id) работает идеально. Множественные коллекции для разных типов памяти (эпизодическая, семантическая, процедурная). **Redis Stack —** если агент требует мгновенных ответов (<10ms) и память помещается в RAM. **LanceDB —** для embedded агентов (локальные, мобильные).

7. Агент постоянно добавляет новые воспоминания, какая БД не деградирует?

Быстрые вставки критичны для агентов. Здесь лучше всего подойдет **Qdrant** (множественные сегменты), **Redis** (in-memory скорость), **Milvus** (быстрая индексация

IVF). Но в целом **Streaming Test** в VectorDBBench покажет реальную производительность при параллельных вставках и поиске.

Итоги

Надеюсь, вам было полезно. Выбор векторной базы для тяжелого прода — это не только выбор базы, но и декомпозиция задачи, выбор эмбедера и много экспериментов.

Спасибо!

Мой крафтовый тг-канальчик Agentic World и другие статьи:

- Порулить браузером через LLM: пишем AI-агента в стиле «browser-use» на ванильной LLM без фреймворков
- Бенчмарк качества распознавания речи (ASR) в телефонии: как мы сравниваемся с Whisper, GigaAM и T-One
- От LangChain к LangGraph: детально разбираемся с фреймворками и всей Lang-экосистемой

Теги: базы данных, milvus, pgvector, qdrant, redis, chroma, chromadb, ии-агенты, ai-агенты, LLM

Хабы: Базы данных, Искусственный интеллект, Машинное обучение, Анализ и проектирование систем

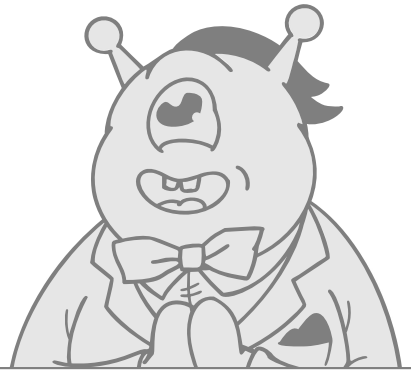
Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Электропочта

Подписаться

Оставляя почту, я принимаю Политику конфиденциальности и даю согласие на получение рассылок





63 Карма

76.1 Общий рейтинг

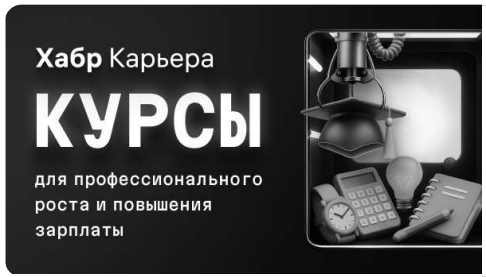
Подписаться

Дмитрий Антипов @antipov_dmitry

AI / LLM / ML / Software

Telegram

Поток AI и ML доступен 24/7 благодаря поддержке друзей Хабра



Хабр Курсы для всех РЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

Перейти

Перейти в поток AI и ML

Комментарии 6

Публикации

ЛУЧШИЕ ЗА СУТКИ | ПОХОЖИЕ

MiracleUsr 7 часов назад

Российский мессенджер MAX замечен в обращении к иностранным сервисам определения IP и серверам конкурентов

Простой | 4 мин | 32K

Аналитика

+97 | 65 | 82 +82

loveprod 9 часов назад

Проект «Прометей»: как озвучить целую библиотеку за один вечер при помощи ИИ

Средний | 8 мин | 5.3K

Обзор

+39 | 23 | 10 +10

ru_vds 8 часов назад

Как получить доступ к российским сайтам из-за рубежа

Средний | 5 мин | 6K

Туториал

+31 23 7 +7



a-yumashin 12 часов назад

Самописный дизайнер форм (WinForms) для VS Code

Средний 7 мин 7.5K

Из песочницы

+29 25 18 +18



interpres 4 часа назад

Одна формула, позволяющая понять 3D-графику

Простой 9 мин 5.2K

Туториал Перевод

+26 26 6 +6



DarthRighter 19 часов назад

ПМК и Кастанеда

Простой 6 мин 8.8K

Из песочницы

+21 14 9 +9

BHV_publishing 9 часов назад

«Анализ данных. Как стать профессионалом». Обзор книги

4 мин 5.5K

+17 26 0

Sivchenko_translate 22 часа назад

Assassin’s Creed: Shadows — проливая свет. Художественный обзор и внутреннее устройство игры. Guest Post

20 мин 11K

+17 2 2 +2

FirstJohn 8 часов назад

EXPLAIN ANALYZE: как находить узкие места в запросах PostgreSQL

Простой 23 мин 4.9K

Туториал

+16 45 10 +10

SSP_blog 10 часов назад

Рецензия на книгу «Генеративный ИИ на практике: трансформеры и диффузионные модели»

Простой 6 мин 4.2K

Мнение

+15 12 0

ИИ здесь, ИИ там, ИИ везде? Опрос

Турбо

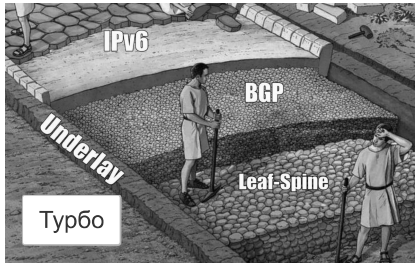
Показать еще

МИНУТОЧКУ ВНИМАНИЯ



Событие

Врываемся в новый год и летим на митапы из Календаря



Турбо

Проектируем Underlay-сеть: BGP, IPv6 и Leaf-Spine



Промо

ИТ-бренд + внешние коммуникации = удержание

ВАКАНСИИ

Разработчик баз данных

от 100 000 до 130 000 ₽ · МегаФон · Москва · Можно удаленно

Разработчик баз данных Middle+/Senior

до 250 000 ₽ · Greenway Global · Новосибирск · Можно удаленно

Инженер-универсал (junior+ – middle-, RPA + AI Agent, Python)

от 80 000 ₽ · СпецЭкономЭнерго · Череповец

ML Data engineer (LLM Gigachat)

от 200 000 ₽ · Сбер · Москва

QA Automation Engineer (Java) mobile

от 300 000 до 450 000 ₽ · Юзтех · Можно удаленно

Больше вакансий на Хабр Карьере

© 2006–2026, Habr